

李萌 2023 011305

Date.

第一章 操作系统引论

一、名词解释

1. 操作系统 是配置在计算机硬件上的第一层软件, 是对硬件系统的首次扩充, 其主要作用是管理硬件设备, 提高它们的利用率和吞吐量, 并为用户和应用程序提供一个简单的接口, 以便于用户和应用程序使用硬件设备。

4. 实时操作系统 是能在确定的时间内对外部事件作出响应并进行处理的操作系统, 强调系统的实时性。

5. 互斥共享 是指在操作系统中, 某些资源在同一时间只能被一个进程访问, 即多个进程对该类资源的访问具有互斥性。

三、填空题

1. 硬件系统 软件系统 3. 硬件资源 软件资源 5 共享性 异步性

第二章

一、名词解释

3. 临界区 是指进程中访问临界资源的那段代码区域

4. 进程同步 是指多个进程在执行过程中, 通过某种机制来协调它们的执行顺序和相对速度, 以保证多个进程能正确、高效地协同工作。

三、填空题

1. 动态性 异步性 结构性 5 动态 静态 6. 间接制约

8. 共享 存储器通信 消息传递通信 管道通信 9 就绪 运行 阻塞

五、问答题

2. S_1 是起始进程, 无前驱; S_2, S_3 的前驱为 S_1 ; S_4, S_5 的前驱为 S_2 ; S_6 的前驱为 S_3 ; S_7 的前驱为 S_4, S_5, S_6

```
#include <stdio.h> void * S1(void * arg) { void * S2(void * arg) {  
#include <pthread.h> printf("S1 is running\n"); sem_wait(&S2_sem);  
#include <semaphore.h> sem_post(&S2_sem); printf("S2 is running\n");  
sem_t S1_sem, S2_sem, S3_sem, S4_sem, S5_sem, S6_sem, S7_sem; sem_post(&S3_sem); sem_post(&S4_sem);  
S4_sem S5_sem, S6_sem, S7_sem; return NULL; } sem_post(&S5_sem);  
return NULL; }
```

// 定义变量

Date.

```

void* S3(void* arg){
    sem_wait(&S3-sem);
    printf("S3 is running\n");
    sem_post(&S6-sem);
    return NULL;}

void* S4(void* arg){
    sem_wait(&S4-sem);
    printf("S4 is running\n");
    sem_post(&S7-sem);
    return NULL;}

void* S5(void* arg){
    sem_wait(&S5-sem);
    printf("S5 is running\n");
    sem_post(&S7-sem);
    return NULL;}

void* S6(void* arg){
    sem_wait(&S6-sem);
    printf("S6 is running\n");
    sem_post(&S7-sem);
    return NULL;}

void* S7(void* arg){
    sem_wait(&S7-sem);
    sem_wait(&S5-sem);
    sem_wait(&S6-sem);
    printf("S7 is running\n");
    return NULL;}

int main() {
    pthread_t t1, t2, ..., t7;
    sem_init(&S1-sem, 0, 0); // 初始化信号量
    ...
    sem_init(&S7-sem, 0, 0);
    pthread_create(&t1, NULL, S1, NULL);
    ... // 创建线程
    pthread_create(&t7, NULL, S7, NULL);
    pthread_join(t1, NULL);
    ...
    pthread_join(t7, NULL);
    sem_destroy(&S1-sem); // 销毁信号量
    ...
    sem_destroy(&S7-sem);
    return 0;
}

```


3. semaphore empty=3;

semaphore apple=0;

semaphore orange=0;

semaphore mutex=1;

void father () {

while (1) {

int fruit Type; // 0代表苹果, 1代表橘子

sem_wait (&empty);

sem_wait (&mutex);

if (fruit Type == 0) {

sem_post (&apple);

} else {

sem_post (&orange);

}

sem_post (&mutex);

}}

void son () {

while (1) {

sem_wait (&orange);

sem_wait (&mutex);

sem_post (&empty);

sem_post (&mutex);

}}

void daughter () {

while (1) {

sem_wait (&apple);

sem_wait (&mutex);

sem_post (&empty);

sem_post (&mutex);

}}

4. 文件打印问题

semaphore full1=0;

semaphore empty1=1;

semaphore full2=0;

semaphore empty2=1;

void PA () {

while (1) {

sem_wait (&empty1);

sem_post (&full1);

}}

void PB () {

while (1) {

sem_wait (&full1);

sem_wait (&empty2);

sem_post (&empty1);

sem_post (&full2);

void PC () {

while (1) {

sem_wait (&full2);

// 从缓冲区2取出记录打印

sem_post (&empty2);

}}

5. 理发师问题

```
semaphore customers = 0;
```

```
semaphore barbers = 0;
```

```
semaphore mutex = 1;
```

```
semaphore payment = 0;
```

```
int waiting = 0;
```

```
int chairs = N;
```

```
void customer() {
```

```
    sem_wait(&mutex);
```

```
    if (waiting < chairs) {
```

```
        waiting ++;
```

```
        sem_post(&customers);
```

```
        sem_post(&mutex);
```

```
        sem_wait(&barbers);
```

```
        sem_wait(&mutex);
```

```
        waiting --;
```

```
        sem_post(&mutex);
```

```
        sem_post(&payment);
```

```
    } else {
```

```
        sem_post(&mutex);
```

```
        // 离开理发店
```

```
    } }
```

```
void barber() {
```

```
    while (1) {
```

```
        sem_wait(&customers);
```

```
        sem_wait(&mutex);
```

```
        waiting --;
```

```
        sem_post(&mutex);
```

```
        sem_post(&barbers);
```

```
        sem_wait(&payment);
```

```
    } }
```


第三章 处理机的调度与死锁

2. 处理

一、名词解释

2. 处理机调度 是指操作系统根据一定的策略和算法, 对系统中的多个进程(或线程)分配处理机资源, 决定哪个进程(或线程)何时获得处理机并投入运行等的过程。

3. 周转时间 是指作业提交给系统开始, 到作业完成为止的这段时间间隔。

4. 死锁 是指多个进程在运行过程中因争夺资源而造成一种僵局。

三、填空题

2. 提交 后备 完成

6. 死锁检测 死锁预防 死锁解除, 避免死锁

松弛度 = 任务截止时间 - 当前时间 - 执行时间, 截止时间 = 周期起点 + 周期, 按松弛度从小到大调度。

初始时刻 (0 ms):

$$A: \text{松弛度} = 20 - 0 - 10 = 10 \text{ ms}$$

$$B: \text{松弛度} = 50 - 0 - 10 = 40 \text{ ms}$$

$$C: \text{松弛度} = 50 - 0 - 15 = 35 \text{ ms}$$

10 ms 时刻:

$$A: \text{松弛度} = 40 - 10 - 10 = 20 \text{ ms}$$

$$B: \text{松弛度} = 50 - 10 - 10 = 30 \text{ ms}$$

$$C: \text{松弛度} = 50 - 10 - 15 = 25 \text{ ms}$$

20 ms

$$A: \text{松弛度} = 60 - 20 - 10 = 30 \text{ ms}$$

$$B: \text{松弛度} = 50 - 20 - 10 = 20 \text{ ms}$$

$$C: \text{松弛度} = 50 - 20 - 15 = 15 \text{ ms}$$

A完成, 更新A下轮截止时间 60ms

C完成, 更新C下轮截止时间 100ms

Date.

35ms 时刻

$$A: \text{松弛度} = 60 - 35 - 10 = 15 \text{ ms}$$

B 完成更新 B 下轮 截止时间 100 ms

$$B: \text{松弛度} = 50 - 35 - 10 = 5 \text{ ms}$$

45ms 时刻

$$A: \text{松弛度} = 60 - 45 - 10 = 5 \text{ ms}$$

$$B: \text{松弛度} = 100 - 45 - 10 = 45 \text{ ms} \quad A \text{ 完成, 更新 A 下轮 截止时间 } 80 \text{ ms}$$

$$C: \text{松弛度} = 100 - 45 - 15 = 40 \text{ ms}$$

后续循环类似, 按周期和松弛度动态调度

3. 银行家算法

$$\text{Allocation} = \text{Max} - \text{Need}$$

$$P_0: (0, 0, 4, 4) - (0, 0, 1, 2) = (0, 0, 3, 2)$$

$$P_1: (2, 7, 5, 0) - (1, 7, 5, 0) = (1, 0, 0, 0)$$

$$P_2: (3, 6, 10, 10) - (2, 3, 5, 6) = (1, 3, 5, 4)$$

$$P_3: (0, 9, 8, 4) - (0, 6, 5, 2) = (0, 3, 3, 2)$$

$$P_4: (0, 6, 6, 10) - (0, 6, 5, 6) = (0, 0, 1, 4)$$

安全性检查: 找 $\text{Need} \leq \text{Available}$ (1, 6, 2, 2) 的进程, P_0 的 Need (0, 0, 1, 2) $\leq$ (1, 6, 2, 2), 分配后回收 P_0 资源, Available 变为 (1+0, 6+0, 2+3, 2+2) = (1, 6, 5, 4); 接着, P_3 的 Need (0, 6, 5, 2) $\leq$ (1, 6, 5, 4), 回收后 $\text{Available} = (1, 9, 8, 6)$; 然后 P_1 的 Need (1, 7, 5, 0), 7 > 9 不行; P_4 的 Need (0, 6, 5, 6) $\leq$ (1, 9, 8, 6), 回收后 $\text{Available} = (1+0, 9+0, 8+1, 6+4) = (1, 9, 9, 10)$; P_2 的 Need (2, 3, 5, 6) $\leq$ (1, 9, 9, 10), 回收后 $\text{Available} = (1+1, 9+3, 9+5, 10+4) = (2, 12, 14, 14)$ 。存在安全序列 $P_0 \rightarrow P_3 \rightarrow P_4 \rightarrow P_2 \rightarrow P_1$, 状态安全。

12) 请求

先检查 $\text{Request} (1, 2, 2, 2) \leq \text{Need} (2, 3, 5, 6)$ (是) 且 $\leq \text{Available} (1, 6, 2, 2)$

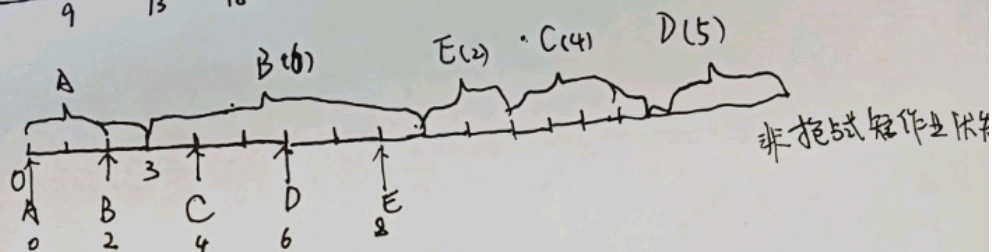
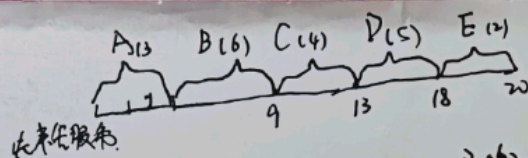
假设分配更新: P_2 的 $\text{Need} = (2-1, 3-2, 5-2, 6-2) = (1, 1, 3, 4)$, $\text{Available} = (1-1, 6-2, 2-2, 2-2) = (0, 4, 0, 0)$ 。

不能分配,预分配后安全性检查无法找到安全序列

(3) 不立即进入死锁状态,只是进入不安全状态。死锁是进程等待资源且无法推进。满足请求后,进程并未处于无限等待资源的情况,只是后续会发生死锁。



东北师范大学
NORTHEAST NORMAL UNIVERSITY
CHINA



答:

1. 先来先服务 (FCFS)

调度顺序 $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$

完成时间: A为3, B为 $3+6=9$, C为 $9+4=13$, D: $13+5=18$, E: $18+2=20$

周转时间: A: $3-0=3$, B: $9-2=7$, C: $13-4=9$, D: $18-6=12$, E: $20-8=12$

带权平均周转时间: $3 \div 3 = 1$, B: $7 \div 6 = 1.17$, C: $9 \div 4 = 2.25$, D: $12 \div 5 = 2.4$, E: $12 \div 2 = 6$ ✓

平均周转时间 $(3+7+9+12+12) \div 5 = 8.6$ 平均带权周转时间 $(1+1.17+2.25+2.4+6) \div 5 = 2.56$

2. 非抢占式短作业优先 (SJF)

调度顺序 $A \rightarrow B \rightarrow E \rightarrow C \rightarrow D$

完成时间: A: 3, B: $3+6=9$, E为 $9+2=11$, C: $11+4=15$, D: $15+5=20$

周转时间: A: $3-0=3$, B: $9-2=7$, E: $11-8=3$, C: $15-4=11$, D: $20-6=14$

带权周转时间: A: $3 \div 3 = 1$, B: $7 \div 6 = 1.17$, E: $3 \div 2 = 1.5$, C: $11 \div 4 = 2.75$, D: $14 \div 5 = 2.8$

平均周转时间: $(3+7+3+11+14) \div 5 = 7.6$ 平均带权周转时间 $(1+1.17+1.5+2.75+2.8) \div 5 = 1.844$

带权